

Contribution Title

First Author¹[0000–1111–2222–3333], Second Author^{2,3}[1111–2222–3333–4444], and
Third Author³[2222–3333–4444–5555]

¹ Princeton University, Princeton NJ 08544, USA

² Springer Heidelberg, Tiergartenstr. 17, 69121 Heidelberg, Germany
lncs@springer.com

<http://www.springer.com/gp/computer-science/lncs>

³ ABC Institute, Rupert-Karls-University Heidelberg, Heidelberg, Germany
{abc,lncs}@uni-heidelberg.de

Tóm tắt nội dung The abstract should briefly summarize the contents
of the paper in 150–250 words.

Keywords: First keyword · Second keyword · Another keyword.

1 Introduction

2 Technical Background

2.1 Problem Definition

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lỗi dựa trên bề mặt vật cản, ta chuyển đổi không gian cấu hình tự do $\mathcal{C}_{\text{free}}$ thành hợp của một tập hữu hạn các vùng lỗi bị chặn. Trong mô hình này, các vùng lỗi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lồi nguyên hỗn hợp (Mixed-Integer Convex Program – MICP). Các biến nguyên được sử dụng để gán từng đoạn quỹ đạo vào các vùng lỗi an toàn tương ứng; số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng trong phân hoạch – không phụ thuộc vào số mặt của vật cản. Nhờ đó phương pháp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các vùng lỗi đảm bảo tính an toàn liên tục theo thời gian.

Để giải quyết thách thức tính toán của bước phân hoạch, một số phương pháp đã được phát triển. Dưới đây là ba phương pháp được sử dụng:

2.2 Phân hoạch Lồi Xấp xỉ (ACD)

Phương pháp Approximate Convex Decomposition (ACD) giới thiệu một cách tiếp cận khác biệt để quản lý sự phức tạp của không gian phi lồi. Thay vì yêu cầu tính lồi hoàn hảo, ACD cho phép các thành phần *lồi xấp xỉ* trong một dung sai do người dùng xác định τ [?]. Điều này thường dẫn đến các phân hoạch có ít thành phần hơn nhiều và phù hợp hơn với các đặc điểm cấu trúc nổi bật của hình học.

Các Khái niệm Cơ bản trong Phân hoạch Lồi Phần này trình bày các khái niệm hình học cơ bản được sử dụng trong quá trình phân hoạch đa giác không lỗi thành các vùng lỗi. Các định nghĩa này đóng vai trò nền tảng cho các phương pháp đo độ lõm và các thuật toán phân hoạch ở các phần sau.

1. **Bao lồi (Convex Hull – H_P):** Bao lồi của một đa giác P là **tập hợp lồi nhỏ nhất chứa toàn bộ đa giác** đó. Trực quan, có thể hình dung bao lồi như một sợi dây chun được kéo căng bao quanh đa giác. Một đa giác P được gọi là **lồi** nếu và chỉ nếu nó trùng với bao lồi của chính nó:

$$P = H_P.$$

2. **Đỉnh lõm (Notch):** Là các đỉnh của đa giác P **không nằm trên bao lồi** H_P . Về mặt hình học, đây là những đỉnh có **góc trong lớn hơn 180°** . Các đỉnh lõm biểu hiện các đặc trưng lõm (concave features) trên biên của đa giác.
3. **Cầu (Bridge):** Là các cạnh của bao lồi ∂H_P nối hai đỉnh không liên kề của biên ngoài ∂P_0 . Tập hợp các cầu được định nghĩa:

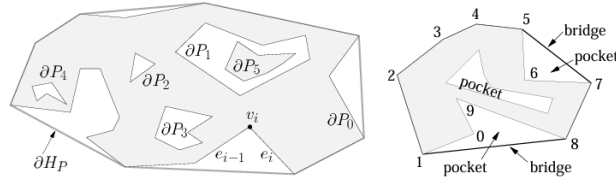
$$\text{BRIDGES}(P) = \partial H_P \setminus \partial P.$$

Mỗi cầu “bắc qua” một vùng lõm của đa giác.

4. **Túi (Pocket):** Là các **chuỗi cạnh liên tiếp** của biên đa giác ∂P không thuộc bao lồi ∂H_P :

$$\text{POCKETS}(P) = \partial P \setminus \partial H_P.$$

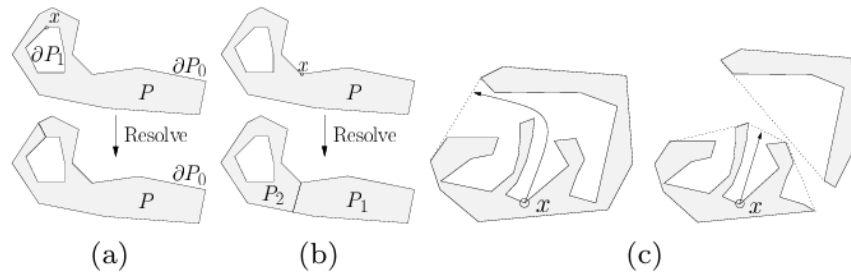
Mỗi túi tương ứng với một “vịnh lõm” (concave bay) trên đường biên của đa giác.



Hình 1. Minh họa bao lồi H_P , đỉnh lõm (notch), cầu (bridge) và túi (pocket) của một đa giác không lồi P .

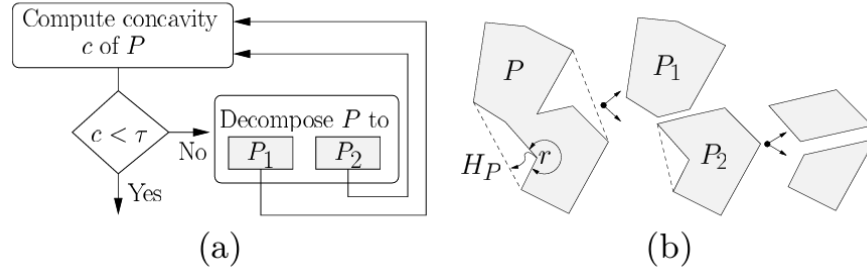
Ý tưởng và Chi tiết Thuật toán Thuật toán ACD là một chiến lược chia để trị đệ quy [?]:

1. **Đo độ lõm:** Xác định đỉnh lõm quan trọng nhất trong đa giác – đỉnh có độ lõm lớn nhất. Các thước đo độ lõm khác nhau sẽ được trình bày ở mục 3.1.
2. **Kiểm tra dung sai:** Nếu độ lõm tối đa nhỏ hơn τ , quá trình kết thúc đối với thành phần đó.
3. **Giải quyết đỉnh lõm:** Nếu độ lõm vượt τ , một đường cắt được thêm vào để giải quyết đỉnh lõm, chia đa giác thành hai thành phần mới (hoặc hợp nhất một lỗ).



Hình 2. (a) Nếu $x \in \partial P_i$ với $i > 0$ (biên của lỗ), hàm **Resolve** gộp lỗ P_i vào đa giác ngoài P_0 . (b) Nếu $x \in \partial P_0$ (biên ngoài), hàm **Resolve** tách đa giác P thành hai thành phần P_1 và P_2 . (c) **Độ lõm** tại điểm x thay đổi sau khi đa giác được phân hoạch.

4. **Đệ quy:** Quá trình được áp dụng đệ quy cho các thành phần mới.



Hình 3. Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn dung sai τ do người dùng đặt trước.

Các Thước đo Độ lõm Chất lượng của phân hoạch phụ thuộc rất nhiều vào **cách đo độ lõm** của các đỉnh trong đa giác không lồi. Các phương pháp đo độ lõm phổ biến bao gồm:

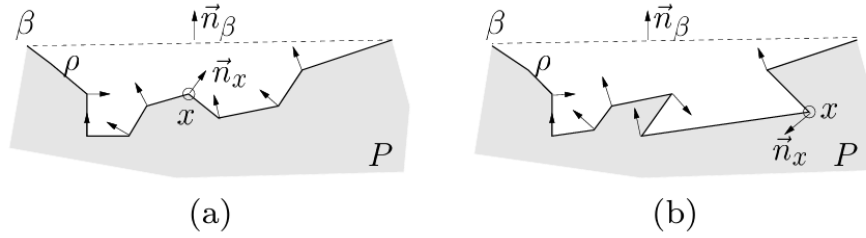
1. **Độ lõm Đường thẳng (Straight-Line Concavity – SL-Concavity):** Là khoảng cách Euclid từ một đỉnh trong *túi* (pocket) đến *cầu* (bridge) liên kết của nó. Phương pháp này rất nhanh, nhưng **không đảm bảo tính đơn điệu**, có thể khiến một số đặc trưng lõm quan trọng bị bỏ sót.



Hình 4. Giả sử r là đỉnh lõm có độ lõm lớn nhất. Sau khi giải quyết r , độ lõm của s tăng lên. Nếu $\text{concavity}(r) < \tau$, đỉnh s sẽ không bao giờ được xử lý, ngay cả khi $\text{concavity}(s) > \tau$.

2. **Độ lõm Đường đi Ngắn nhất (Shortest Path Concavity – SP-Concavity):** Là **chiều dài đường đi ngắn nhất** từ một đỉnh trong túi đến cầu tương ứng, với điều kiện đường đi phải nằm hoàn toàn trong túi. Phương pháp này chậm hơn SL-Concavity nhưng **đảm bảo tính đơn điệu**: độ lõm của các đỉnh giảm dần sau mỗi lần phân hoạch, giúp đảm bảo tất cả các đặc trưng lõm vượt ngưỡng τ đều được xử lý.
3. **Độ lõm Lai (Hybrid Concavity – H-Concavity):** Một giải pháp trung gian thực tiễn: sử dụng SL-Concavity nhanh làm mặc định và chỉ chuyển

sang SP-Concavity khi **phát hiện khả năng không đơn điệu**. Bước kiểm tra dựa trên so sánh pháp tuyến n_β của cầu và pháp tuyến n_i của đỉnh trong túi: nếu $n_i \cdot n_\beta < 0$, biên túi bị đảo hướng – dấu hiệu của túi phức tạp mà SL-Concavity có thể thất bại.



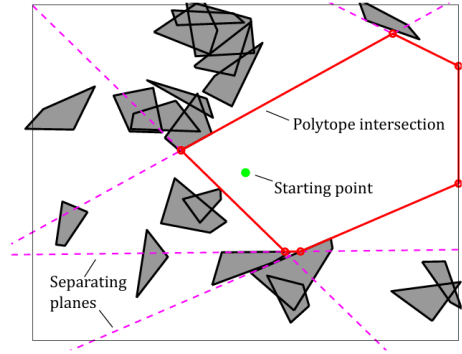
Hình 5. SL-Concavity xử lý chính xác túi (a) vì không có pháp tuyến đỉnh nào trong túi ngược chiều pháp tuyến cầu. Tuy nhiên, túi (b) có thể dẫn đến độ lõm giảm không đơn điệu, khiến SL-Concavity thất bại.

2.3 Iterative Regional Inflation by Semidefinite Programming (IRIS)

Thay vì cố gắng bao phủ toàn bộ không gian tự do cùng một lúc, thuật toán IRIS (Iterative Regional Inflation by Semidefinite Programming) đặt ra câu hỏi: “Với một điểm mầm (*seed*) cho trước, đa diện lồi lớn nhất không va chạm với bất kỳ vật cản nào xung quanh nó là gì?” Mỗi lần gọi thuật toán cho ra một đa diện lồi từ một điểm mầm duy nhất; để phủ toàn bộ C_{free} , thuật toán được chạy nhiều lần với các điểm mầm khác nhau.

Ý tưởng và Chi tiết Thuật toán IRIS là một quy trình tối ưu hóa lập gồm bốn bước:

1. **Khởi tạo:** Thuật toán tạo một ellipsoid ban đầu là hình cầu rất nhỏ có tâm tại điểm mầm.
2. **Tìm Siêu phẳng Phân cách (QP):**
Đối với ellipsoid hiện tại, thuật toán tìm điểm gần nhất trên mỗi vật cản, sau đó xây dựng siêu phẳng phân cách ellipsoid khỏi các vật cản đó. Mỗi siêu phẳng tiếp tuyến với ellipsoid được mở rộng và đi qua điểm gần nhất trên vật cản tương ứng. Các siêu phẳng thừa (không xác định biên của vùng lồi kết quả) được loại bỏ để giữ quy mô bài toán nhỏ. Bước này gồm bốn thao tác nhỏ:
 1. **Biến đổi về không gian cầu đơn vị:** Việc tính khoảng cách đến ellipsoid phức tạp hơn so với cầu đơn vị. Ellipsoid E được định nghĩa là ảnh của cầu đơn vị qua phép biến đổi affine $x = C\tilde{x} + d$ (với C xác định hình dạng và hướng, d là tâm). Áp dụng phép biến đổi ngược



Hình 6. Minh họa siêu phẳng phân cách ellipsoid khỏi các vật cản trong IRIS.

$\tilde{x} = C^{-1}(x - d)$ để đưa mọi điểm về không gian cầu đơn vị; khi đó mỗi vật cản l_j được biến đổi thành $\tilde{l}_j$.

2. **Tìm điểm gần nhất bằng Quy hoạch Toàn phương (QP):** Tìm điểm trên vật cản l_j gần ellipsoid E nhất tương đương với tìm điểm trên $\tilde{l}_j$ gần gốc tọa độ nhất:

$$\begin{aligned} & \underset{\tilde{x} \in \mathbb{R}^n, w \in \mathbb{R}^m}{\text{minimize}} && \|\tilde{x}\|^2 \\ & \text{subject to} && \begin{cases} \begin{bmatrix} \tilde{v}_{j,1} & \tilde{v}_{j,2} & \cdots & \tilde{v}_{j,m} \end{bmatrix} w = \tilde{x}, \\ \sum_{i=1}^m w_i = 1, \\ w_i \geq 0, \quad i = 1, \dots, m. \end{cases} \end{aligned}$$

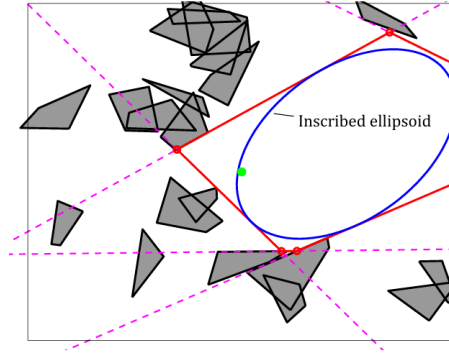
Về bản chất, ta tìm **tổ hợp lồi** của các đỉnh đã biến đổi $\tilde{v}_{j,k}$ gần gốc tọa độ nhất.

3. **Xây dựng siêu phẳng tiếp tuyến:** Sau khi tìm được điểm x^* gần nhất, siêu phẳng tiếp tuyến với ellipsoid tại x^* được xây dựng. Ellipsoid có dạng $E = \{x \mid (x - d)^\top C^{-1} C^{-\top} (x - d) \leq 1\}$, nên pháp tuyến tại x^* là:

$$a_j = 2C^{-1}C^{-\top}(x^* - d), \quad b_j = a_j^\top x^*.$$

Siêu phẳng $a_j^\top x = b_j$ được thêm vào như một ràng buộc tuyến tính, đảm bảo ellipsoid mở rộng không giao với vật cản tương ứng.

4. **Lập lại với tất cả vật cản:** Thu được tập siêu phẳng xác định một đa diện lồi (polytope) không giao với bất kỳ vật cản nào.
3. **Tìm Ellipsoid Nội tiếp Cực đại (SDP):** Giao của các nửa không gian từ bước 2 tạo thành một đa diện lồi $P = \{x \in \mathbb{R}^n \mid Ax \leq b\}$. Thuật toán sau đó tìm ellipsoid có thể tích lớn nhất nội tiếp trong P .



Hình 7. Ellipsoid nội tiếp cực đại trong đa diện lồi hiện tại.

Ellipsoid được tham số hóa là $\mathcal{E} = \{C\tilde{x} + d \mid \|\tilde{x}\|_2 \leq 1\}$ với thể tích tỉ lệ $\det(C)$. Bài toán tối ưu lồi tương ứng là:

$$\begin{aligned} & \underset{C, d}{\text{maximize}} && \log \det C \\ & \text{subject to} && \|a_i^\top C\|_2 + a_i^\top d \leq b_i, \quad \forall i = 1, \dots, N, \\ & && C \succ 0. \end{aligned} \quad (1)$$

Ràng buộc $\|a_i^\top C\|_2 + a_i^\top d \leq b_i$ đảm bảo mọi điểm của ellipsoid đều nằm trong P . Đây là bài toán bán xác định dương (SDP), có thể giải hiệu quả bằng các bộ giải SDP chuẩn.

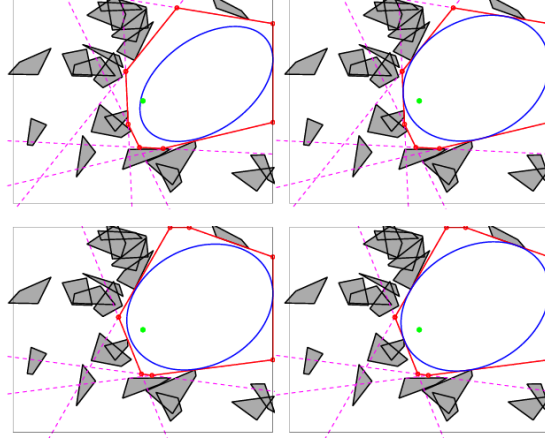
Mục tiêu: Cực đại hóa $\log \det C$ để tối đa hóa thể tích ellipsoid.

Ràng buộc: Đảm bảo mọi điểm của ellipsoid đều nằm trong đa diện P .

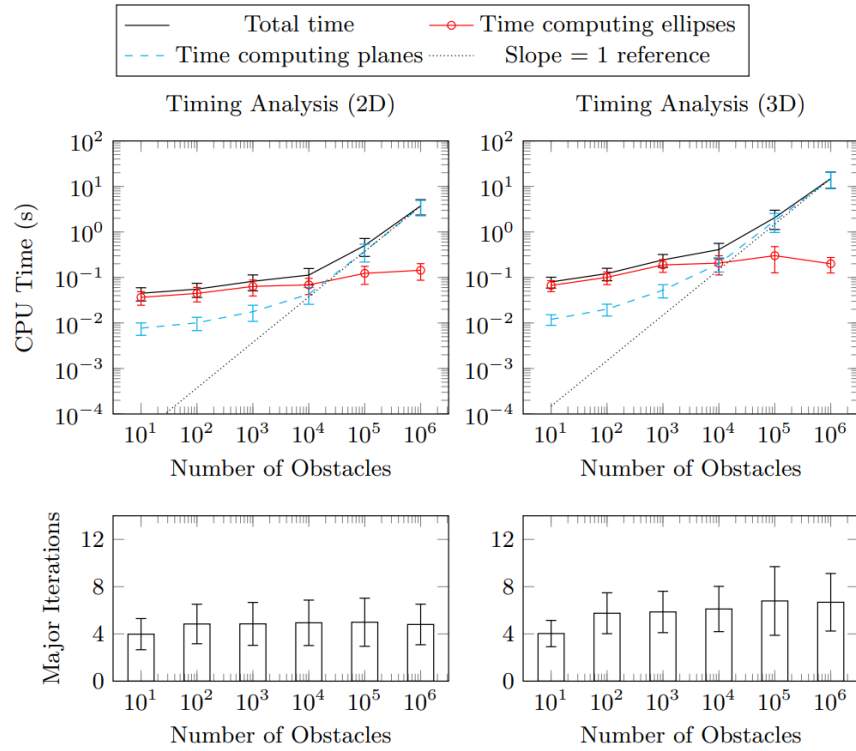
4. **Lặp lại:** Ellipsoid mới, lớn hơn được sử dụng làm điểm khởi đầu cho vòng lặp tiếp theo. Quá trình hội tụ khi thể tích ellipsoid ngừng tăng đáng kể, trả về một đa diện lồi cùng ellipsoid nội tiếp của nó.

Đặc tính của Thuật toán

- **Loại thuật toán:** IRIS là phương pháp xấp xỉ để bao phủ không gian tự do. Một lần chạy chỉ tạo ra một vùng lồi; để phủ hoàn chỉnh C_{free} cần chạy nhiều lần với các điểm mầm khác nhau.
- **Độ phức tạp thời gian:** IRIS thường hội tụ qua 4–8 vòng lặp. Mỗi vòng gồm bước tìm siêu phẳng (QP, tăng tuyến tính theo số vật cản) và bước tìm ellipsoid nội tiếp (SDP, gần như không đổi nhờ loại siêu phẳng thừa).
- **Xử lý vật cản không lồi:**
Trong không gian làm việc (workspace): IRIS ban đầu giả định vật cản là lồi. Để xử lý vật cản không lồi, ta tiền xử lý bằng cách lấy bao lồi hoặc phân hoạch chúng thành các mảnh lồi trước khi chạy IRIS [?]. Điều này khả thi vì IRIS mở rộng hiệu quả theo số lượng vật cản.



Hình 8. Quá trình lặp IRIS hội tụ sau 4–8 vòng trong hầu hết trường hợp.

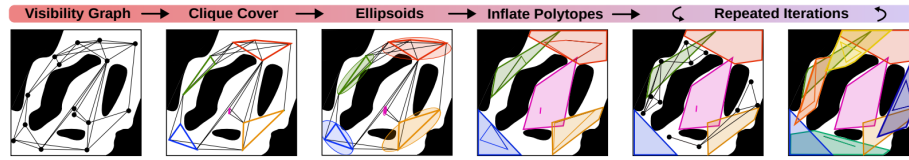


Hình 9. Độ phức tạp thời gian của IRIS [?].

Trong không gian cấu hình (configuration space): Đối với robot có động học phức tạp, các phần mở rộng như IRIS-NP [?] và C-IRIS [?] được sử dụng để xử lý ràng buộc va chạm được định nghĩa ẩn và không lỗi.

2.4 Visibility Clique Cover (VCC)

Thuật toán Visibility Clique Cover (VCC) giải quyết trực tiếp một hạn chế chính của IRIS: sự phụ thuộc vào chất lượng điểm mẫu [?]. Việc lấy mẫu ngẫu nhiên thường không hiệu quả. VCC tự động hóa quá trình này bằng cách trước tiên tìm hiểu cấu trúc toàn cục của không gian tự do, sau đó đặt điểm mẫu có chủ đích vào các vùng có khả năng mở rộng lớn.



Hình 10. Tổng quan về quy trình VCC [?].

Ý tưởng và Chi tiết Thuật toán Quy trình VCC như minh họa trong Hình 20 bao gồm năm bước chính:

1. **Lấy mẫu và xây dựng đồ thị khả kiến:** Thuật toán lấy mẫu n điểm ngẫu nhiên trong $\mathcal{C}_{\text{free}}$. Các điểm này tạo thành đỉnh của một đồ thị khả kiến (visibility graph)⁴.
2. **Tìm Clique Cover:** Thuật toán tìm một tập hợp các clique lớn (đồ thị con đầy đủ) trong đồ thị khả kiến. Ý tưởng cốt lõi: tập điểm mà tất cả đều “nhìn thấy” nhau có khả năng cao nằm trong cùng một vùng lõm⁵. Clique cover được tìm bằng thuật toán tham lam lặp lại, trong đó mỗi vòng lặp giải gần đúng bài toán MAXCLIQUE thường được xây dựng dưới dạng Integer Linear Programming (ILP).
3. **Khởi tạo Ellipsoid:** Với mỗi clique tìm được, thuật toán tính ellipsoid có thể tích nhỏ nhất bao quanh tất cả các điểm trong clique đó. Ellipsoid này cung cấp tâm (điểm mẫu) và hình dạng ban đầu cho bước lấp đầy tiếp theo.
4. **Lấp đầy thành Đa diện:** Tâm và hình dạng của mỗi ellipsoid khởi tạo một vòng lặp IRIS duy nhất. Nhờ ellipsoid đã được định hướng bởi hình học cục bộ, bước lấp đầy thường chỉ cần một vòng lặp – hiệu quả hơn đáng kể so với IRIS tiêu chuẩn [?].

⁴ *Visibility graph* là đồ thị trong đó có cạnh nối hai đỉnh nếu và chỉ nếu đoạn thẳng nối chúng nằm hoàn toàn trong $\mathcal{C}_{\text{free}}$, tức là không va chạm với bất kỳ vật cản nào.

⁵ Nếu hai điểm nhìn thấy nhau, đoạn thẳng nối chúng nằm trong $\mathcal{C}_{\text{free}}$ – tính chất đặc trưng của tập lõm.

5. **Lắp lại và hội tụ:** Quá trình được lắp lại bằng cách lấy mẫu mới từ không gian chưa được phủ cho đến khi đạt độ phủ mong muốn.

Đặc tính của Thuật toán

- **Tự động hóa đặt điểm mầm:** Khác với IRIS yêu cầu lựa chọn điểm mầm thủ công, VCC tự suy ra vị trí đặt mầm từ cấu trúc đồ thị khả kiến, giảm thiểu sự phụ thuộc vào kiến thức chuyên gia về môi trường.
- **Độ phức tạp tính toán:** Bài toán MAXCLIQUE là NP-hard trong trường hợp tổng quát; VCC dùng thuật toán tham lam lắp lại để giải gần đúng, cho kết quả tốt trong thực tế với thời gian đa thức đối với hầu hết cấu hình môi trường. Giai đoạn lấp đầy đa diện tương đương một vòng lặp IRIS với khối tạo tốt, nên nhanh hơn IRIS tiêu chuẩn đáng kể.
- **Chất lượng phân hoạch:** VCC có xu hướng tạo ít vùng hơn để phủ cùng một không gian so với IRIS lấy mầm ngẫu nhiên, nhờ điểm mầm được đặt có chủ đích dựa trên cấu trúc hình học. Tuy nhiên, chất lượng phụ thuộc vào số lượng mẫu n và tính đại diện của đồ thị khả kiến.
- **Giới hạn:** VCC yêu cầu lấy mẫu đủ dày để xây dựng đồ thị khả kiến đại diện. Trong các không gian hẹp (narrow passages) hoặc không gian chiều cao, đây có thể là thách thức đáng kể.

Tổng kết và lựa chọn phương pháp

Ba phương pháp trình bày ở trên có những ưu nhược điểm bổ sung cho nhau, được tóm tắt trong Bảng 2.

Bảng 1. So sánh các phương pháp phân hoạch không gian an toàn.

Tiêu chí	ACD	IRIS	VCC
Phủ toàn bộ C_{free}	Có (xấp xỉ τ)	Theo seed	Tự động lắp
Yêu cầu vật cản lồi	Không	Có	Có
Tự động đặt điểm mầm	–	Không	Có
Bộ giải sử dụng	Heuristic	QP + SDP	ILP + QP + SDP
Phù hợp với	Polygon 2D	$\mathbb{R}^n$ tổng quát	$\mathbb{R}^n$ tổng quát

Trong đề án này, bước phân hoạch không gian an toàn được thực hiện bằng một trong các phương pháp trên để tạo ra họ vùng lồi $\{Q_v\}$ làm đầu vào chung cho cả hai hướng tiếp cận GCS-Bézier (Phần ??) và GCS-DMS (Phần ??). Chi tiết về tham số và kết quả phân hoạch trên các môi trường thực nghiệm được trình bày ở Chương ??.

3 Phân hoạch không gian an toàn

Thay vì tiếp cận bài toán tránh vật cản theo cách truyền thống là mô tả các ràng buộc không lồi dựa trên bề mặt vật cản, ta chuyển đổi không gian cấu

hình tự do $\mathcal{C}_{\text{free}}$ thành hợp của một tập hữu hạn các vùng lồi bị chặn. Trong mô hình này, các vùng lồi có thể chồng lấn lên nhau, và bài toán hoạch định chuyển động được phát biểu lại dưới dạng một chương trình quy hoạch lồi nguyên hỗn hợp (Mixed-Integer Convex Program – MICP). Các biến nguyên được sử dụng để gán từng đoạn quỹ đạo vào các vùng lồi an toàn tương ứng; số lượng biến nguyên chỉ phụ thuộc vào số lượng vùng trong phân hoạch – không phụ thuộc vào số mặt của vật cản. Nhờ đó phương pháp giải quyết hiệu quả các môi trường phức tạp với hàng trăm vật cản. Hơn nữa, việc ràng buộc quỹ đạo nằm trọn trong các vùng lồi đảm bảo tính an toàn liên tục theo thời gian.

Để giải quyết thách thức tính toán của bước phân hoạch, một số phương pháp đã được phát triển. Dưới đây là ba phương pháp được sử dụng trong đồ án này:

3.1 Phân hoạch Lồi Xấp xỉ (ACD)

Phương pháp Approximate Convex Decomposition (ACD) giới thiệu một cách tiếp cận khác biệt để quản lý sự phức tạp của không gian phi lồi. Thay vì yêu cầu tính lồi hoàn hảo, ACD cho phép các thành phần *lồi xấp xỉ* trong một dung sai do người dùng xác định τ [?]. Điều này thường dẫn đến các phân hoạch có ít thành phần hơn nhiều và phù hợp hơn với các đặc điểm cấu trúc nổi bật của hình học.

Các Khái niệm Cơ bản trong Phân hoạch Lồi Phần này trình bày các khái niệm hình học cơ bản được sử dụng trong quá trình phân hoạch đa giác không lồi thành các vùng lồi. Các định nghĩa này đóng vai trò nền tảng cho các phương pháp đo độ lõm và các thuật toán phân hoạch ở các phần sau.

1. **Bao lồi (Convex Hull – H_P):** Bao lồi của một đa giác P là **tập hợp lồi nhỏ nhất chứa toàn bộ đa giác** đó. Trực quan, có thể hình dung bao lồi như một sợi dây chun được kéo căng bao quanh đa giác. Một đa giác P được gọi là **lồi** nếu và chỉ nếu nó trùng với bao lồi của chính nó:

$$P = H_P.$$

2. **Đỉnh lõm (Notch):** Là các đỉnh của đa giác P **không nằm trên bao lồi** H_P . Về mặt hình học, đây là những đỉnh có **góc trong lớn hơn 180°** . Các đỉnh lõm biểu hiện các đặc trưng lõm (concave features) trên biên của đa giác.
3. **Cầu (Bridge):** Là các cạnh của bao lồi ∂H_P nối hai đỉnh không liên kề của biên ngoài ∂P_0 . Tập hợp các cầu được định nghĩa:

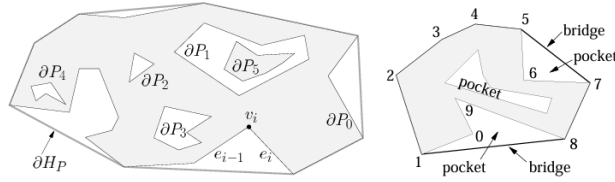
$$\text{BRIDGES}(P) = \partial H_P \setminus \partial P.$$

Mỗi cầu “bắc qua” một vùng lõm của đa giác.

4. **Túi (Pocket):** Là các **chuỗi cạnh liên tiếp** của biên đa giác ∂P không thuộc bao lồi ∂H_P :

$$\text{POCKETS}(P) = \partial P \setminus \partial H_P.$$

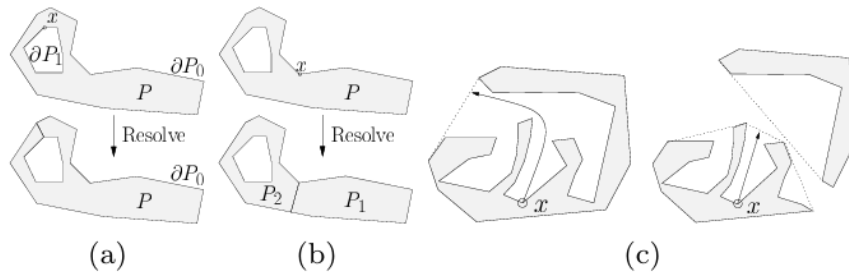
Mỗi túi tương ứng với một “vịnh lõm” (concave bay) trên đường biên của đa giác.



Hình 11. Minh họa bao lồi H_P , đỉnh lõm (notch), cầu (bridge) và túi (pocket) của một đa giác không lồi P .

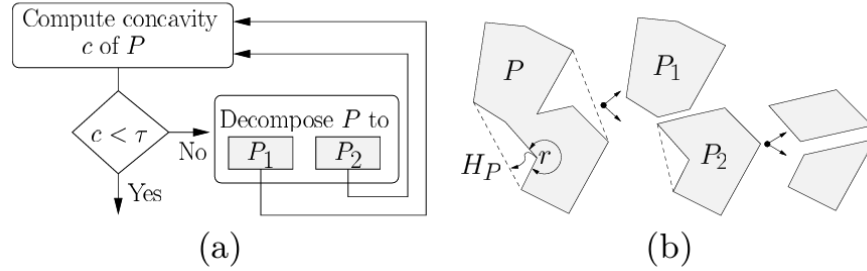
Ý tưởng và Chi tiết Thuật toán Thuật toán ACD là một chiến lược chia để trị đệ quy [?]:

1. **Đo độ lõm:** Xác định đỉnh lõm quan trọng nhất trong đa giác – đỉnh có độ lõm lớn nhất. Các thước đo độ lõm khác nhau sẽ được trình bày ở mục 3.1.
2. **Kiểm tra dung sai:** Nếu độ lõm tối đa nhỏ hơn τ , quá trình kết thúc đối với thành phần đó.
3. **Giải quyết đỉnh lõm:** Nếu độ lõm vượt τ , một đường cắt được thêm vào để giải quyết đỉnh lõm, chia đa giác thành hai thành phần mới (hoặc hợp nhất một lỗ).



Hình 12. (a) Nếu $x \in \partial P_i$ với $i > 0$ (biên của lỗ), hàm **Resolve gộp** lỗ P_i vào đa giác ngoài P_0 . (b) Nếu $x \in \partial P_0$ (biên ngoài), hàm **Resolve tách** đa giác P thành hai thành phần P_1 và P_2 . (c) **Độ lõm** tại điểm x thay đổi sau khi đa giác được phân hoạch.

4. **Đệ quy:** Quá trình được áp dụng đệ quy cho các thành phần mới.



Hình 13. Quá trình đệ quy tiếp tục cho đến khi đạt đến giới hạn dung sai τ do người dùng đặt trước.

Các Thước đo Độ lõm Chất lượng của phân hoạch phụ thuộc rất nhiều vào **cách đo độ lõm** của các đỉnh trong đa giác không lồi. Các phương pháp đo độ lõm phổ biến bao gồm:

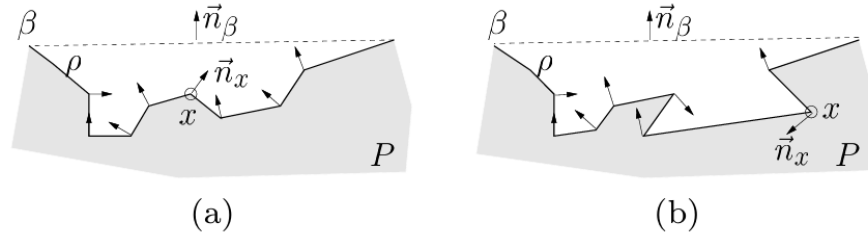
1. **Độ lõm Đường thẳng (Straight-Line Concavity – SL-Concavity):** Là khoảng cách Euclid từ một đỉnh trong *túi* (pocket) đến *cầu* (bridge) liên kết của nó. Phương pháp này rất nhanh, nhưng **không đảm bảo tính đơn điệu**, có thể khiến một số đặc trưng lõm quan trọng bị bỏ sót.



Hình 14. Giả sử r là đỉnh lõm có độ lõm lớn nhất. Sau khi giải quyết r , độ lõm của s tăng lên. Nếu $\text{concavity}(r) < \tau$, đỉnh s sẽ không bao giờ được xử lý, ngay cả khi $\text{concavity}(s) > \tau$.

2. **Độ lõm Đường đi Ngắn nhất (Shortest Path Concavity – SP-Concavity):** Là **chiều dài đường đi ngắn nhất** từ một đỉnh trong túi đến cầu tương ứng, với điều kiện đường đi phải nằm hoàn toàn trong túi. Phương pháp này chậm hơn SL-Concavity nhưng **đảm bảo tính đơn điệu**: độ lõm của các đỉnh giảm dần sau mỗi lần phân hoạch, giúp đảm bảo tất cả các đặc trưng lõm vượt ngưỡng τ đều được xử lý.
3. **Độ lõm Lai (Hybrid Concavity – H-Concavity):** Một giải pháp trung gian thực tiễn: sử dụng SL-Concavity nhanh làm mặc định và chỉ chuyển

sang SP-Concavity khi **phát hiện khả năng không đơn điệu**. Bước kiểm tra dựa trên so sánh pháp tuyến n_β của cầu và pháp tuyến n_i của đỉnh trong túi: nếu $n_i \cdot n_\beta < 0$, biên túi bị đảo hướng – dấu hiệu của túi phức tạp mà SL-Concavity có thể thất bại.



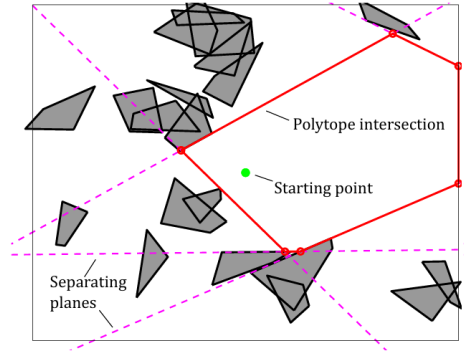
Hình 15. SL-Concavity xử lý chính xác túi (a) vì không có pháp tuyến đỉnh nào trong túi ngược chiều pháp tuyến cầu. Tuy nhiên, túi (b) có thể dẫn đến độ lõm giảm không đơn điệu, khiến SL-Concavity thất bại.

3.2 Iterative Regional Inflation by Semidefinite Programming (IRIS)

Thay vì cố gắng bao phủ toàn bộ không gian tự do cùng một lúc, thuật toán IRIS (Iterative Regional Inflation by Semidefinite Programming) đặt ra câu hỏi: “Với một điểm mầm (*seed*) cho trước, đa diện lồi lớn nhất không va chạm với bất kỳ vật cản nào xung quanh nó là gì?” Mỗi lần gọi thuật toán cho ra một đa diện lồi từ một điểm mầm duy nhất; để phủ toàn bộ C_{free} , thuật toán được chạy nhiều lần với các điểm mầm khác nhau.

Ý tưởng và Chi tiết Thuật toán IRIS là một quy trình tối ưu hóa lập gồm bốn bước:

1. **Khởi tạo:** Thuật toán tạo một ellipsoid ban đầu là hình cầu rất nhỏ có tâm tại điểm mầm.
2. **Tìm Siêu phẳng Phân cách (QP):**
 Đối với ellipsoid hiện tại, thuật toán tìm điểm gần nhất trên mỗi vật cản, sau đó xây dựng siêu phẳng phân cách ellipsoid khỏi các vật cản đó. Mỗi siêu phẳng tiếp tuyến với ellipsoid được mở rộng và đi qua điểm gần nhất trên vật cản tương ứng. Các siêu phẳng thừa (không xác định biên của vùng lồi kết quả) được loại bỏ để giữ quy mô bài toán nhỏ. Bước này gồm bốn thao tác nhỏ:
 1. **Biến đổi về không gian cầu đơn vị:** Việc tính khoảng cách đến ellipsoid phức tạp hơn so với cầu đơn vị. Ellipsoid E được định nghĩa là ảnh của cầu đơn vị qua phép biến đổi affine $x = C\tilde{x} + d$ (với C xác định hình dạng và hướng, d là tâm). Áp dụng phép biến đổi ngược



Hình 16. Minh họa siêu phẳng phân cách ellipsoid khỏi các vật cản trong IRIS.

$\tilde{x} = C^{-1}(x - d)$ để đưa mọi điểm về không gian cầu đơn vị; khi đó mỗi vật cản l_j được biến đổi thành $\tilde{l}_j$.

2. **Tìm điểm gần nhất bằng Quy hoạch Toàn phương (QP):** Tìm điểm trên vật cản l_j gần ellipsoid E nhất tương đương với tìm điểm trên $\tilde{l}_j$ gần gốc tọa độ nhất:

$$\begin{aligned} & \underset{\tilde{x} \in \mathbb{R}^n, w \in \mathbb{R}^m}{\text{minimize}} && \|\tilde{x}\|^2 \\ & \text{subject to} && \begin{cases} \begin{bmatrix} \tilde{v}_{j,1} & \tilde{v}_{j,2} & \cdots & \tilde{v}_{j,m} \end{bmatrix} w = \tilde{x}, \\ \sum_{i=1}^m w_i = 1, \\ w_i \geq 0, \quad i = 1, \dots, m. \end{cases} \end{aligned}$$

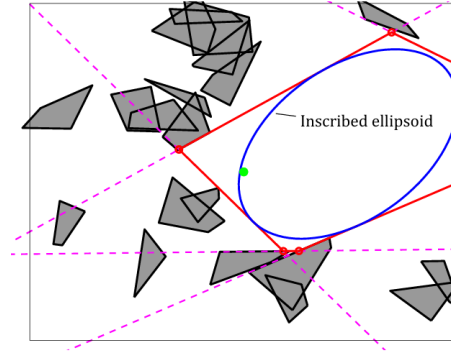
Về bản chất, ta tìm **tổ hợp lồi** của các đỉnh đã biến đổi $\tilde{v}_{j,k}$ gần gốc tọa độ nhất.

3. **Xây dựng siêu phẳng tiếp tuyến:** Sau khi tìm được điểm x^* gần nhất, siêu phẳng tiếp tuyến với ellipsoid tại x^* được xây dựng. Ellipsoid có dạng $E = \{x \mid (x - d)^\top C^{-1} C^{-\top} (x - d) \leq 1\}$, nên pháp tuyến tại x^* là:

$$a_j = 2C^{-1}C^{-\top}(x^* - d), \quad b_j = a_j^\top x^*.$$

Siêu phẳng $a_j^\top x = b_j$ được thêm vào như một ràng buộc tuyến tính, đảm bảo ellipsoid mở rộng không giao với vật cản tương ứng.

4. **Lập lại với tất cả vật cản:** Thu được tập siêu phẳng xác định một đa diện lồi (polytope) không giao với bất kỳ vật cản nào.
3. **Tìm Ellipsoid Nội tiếp Cực đại (SDP):** Giao của các nửa không gian từ bước 2 tạo thành một đa diện lồi $P = \{x \in \mathbb{R}^n \mid Ax \leq b\}$. Thuật toán sau đó tìm ellipsoid có thể tích lớn nhất nội tiếp trong P .



Hình 17. Ellipsoid nội tiếp cực đại trong đa diện lồi hiện tại.

Ellipsoid được tham số hóa là $\mathcal{E} = \{C\tilde{x} + d \mid \|\tilde{x}\|_2 \leq 1\}$ với thể tích tỉ lệ $\det(C)$. Bài toán tối ưu lồi tương ứng là:

$$\begin{aligned} & \underset{C, d}{\text{maximize}} && \log \det C \\ & \text{subject to} && \|a_i^\top C\|_2 + a_i^\top d \leq b_i, \quad \forall i = 1, \dots, N, \\ & && C \succ 0. \end{aligned} \quad (2)$$

Ràng buộc $\|a_i^\top C\|_2 + a_i^\top d \leq b_i$ đảm bảo mọi điểm của ellipsoid đều nằm trong P . Đây là bài toán bán xác định dương (SDP), có thể giải hiệu quả bằng các bộ giải SDP chuẩn.

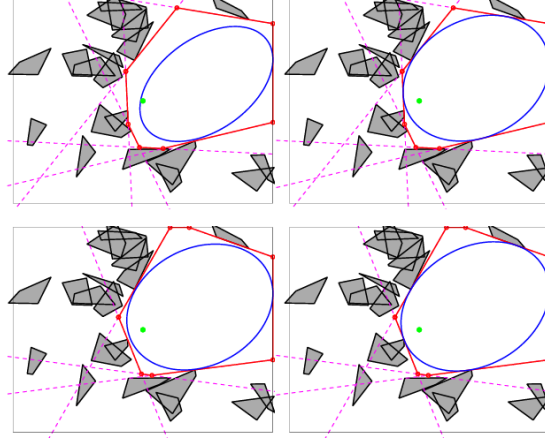
Mục tiêu: Cực đại hóa $\log \det C$ để tối đa hóa thể tích ellipsoid.

Ràng buộc: Đảm bảo mọi điểm của ellipsoid đều nằm trong đa diện P .

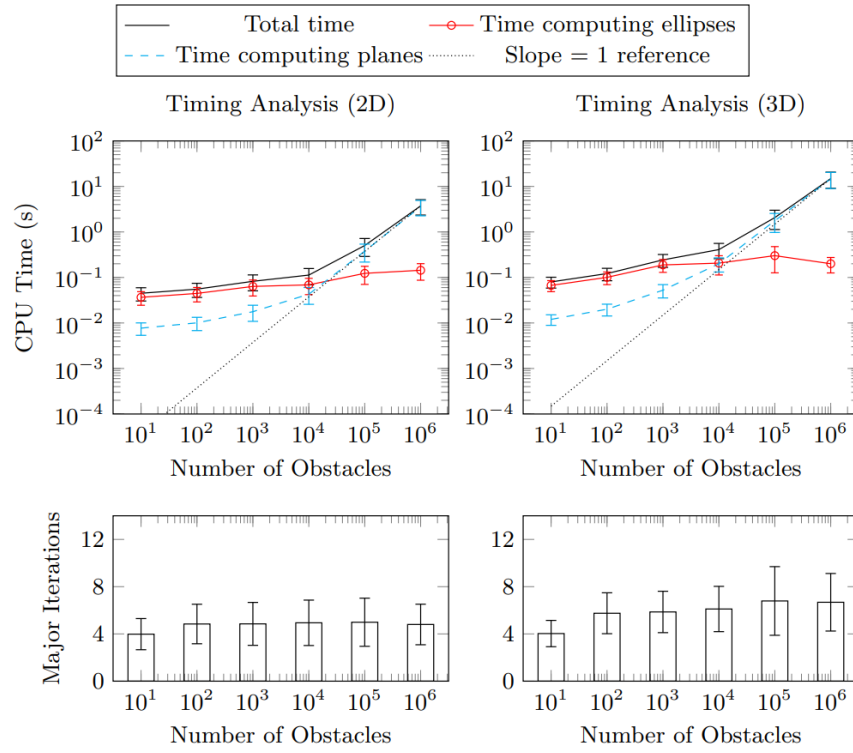
4. **Lặp lại:** Ellipsoid mới, lớn hơn được sử dụng làm điểm khởi đầu cho vòng lặp tiếp theo. Quá trình hội tụ khi thể tích ellipsoid ngừng tăng đáng kể, trả về một đa diện lồi cùng ellipsoid nội tiếp của nó.

Đặc tính của Thuật toán

- **Loại thuật toán:** IRIS là phương pháp xấp xỉ để bao phủ không gian tự do. Một lần chạy chỉ tạo ra một vùng lồi; để phủ hoàn chỉnh C_{free} cần chạy nhiều lần với các điểm mầm khác nhau.
- **Độ phức tạp thời gian:** IRIS thường hội tụ qua 4–8 vòng lặp. Mỗi vòng gồm bước tìm siêu phẳng (QP, tăng tuyến tính theo số vật cản) và bước tìm ellipsoid nội tiếp (SDP, gần như không đổi nhờ loại siêu phẳng thừa).
- **Xử lý vật cản không lồi:**
Trong không gian làm việc (workspace): IRIS ban đầu giả định vật cản là lồi. Để xử lý vật cản không lồi, ta tiền xử lý bằng cách lấy bao lồi hoặc phân hoạch chúng thành các mảnh lồi trước khi chạy IRIS [?]. Điều này khả thi vì IRIS mở rộng hiệu quả theo số lượng vật cản.



Hình 18. Quá trình lặp IRIS hội tụ sau 4–8 vòng trong hầu hết trường hợp.

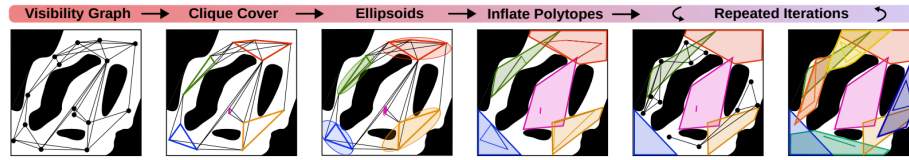


Hình 19. Độ phức tạp thời gian của IRIS [?].

Trong không gian cấu hình (configuration space): Đối với robot có động học phức tạp, các phần mở rộng như IRIS-NP [?] và C-IRIS [?] được sử dụng để xử lý ràng buộc va chạm được định nghĩa ẩn và không lỗi.

3.3 Visibility Clique Cover (VCC)

Thuật toán Visibility Clique Cover (VCC) giải quyết trực tiếp một hạn chế chính của IRIS: sự phụ thuộc vào chất lượng điểm mẫu [?]. Việc lấy mẫu ngẫu nhiên thường không hiệu quả. VCC tự động hóa quá trình này bằng cách trước tiên tìm hiểu cấu trúc toàn cục của không gian tự do, sau đó đặt điểm mẫu có chủ đích vào các vùng có khả năng mở rộng lớn.



Hình 20. Tổng quan về quy trình VCC [?].

Ý tưởng và Chi tiết Thuật toán Quy trình VCC như minh họa trong Hình 20 bao gồm năm bước chính:

1. **Lấy mẫu và xây dựng đồ thị khả kiến:** Thuật toán lấy mẫu n điểm ngẫu nhiên trong $\mathcal{C}_{\text{free}}$. Các điểm này tạo thành đỉnh của một đồ thị khả kiến (visibility graph)⁶.
2. **Tìm Clique Cover:** Thuật toán tìm một tập hợp các clique lớn (đồ thị con đầy đủ) trong đồ thị khả kiến. Ý tưởng cốt lõi: tập điểm mà tất cả đều “nhìn thấy” nhau có khả năng cao nằm trong cùng một vùng lỗi⁷. Clique cover được tìm bằng thuật toán tham lam lặp lại, trong đó mỗi vòng lặp giải gần đúng bài toán MAXCLIQUE thường được xây dựng dưới dạng Integer Linear Programming (ILP).
3. **Khởi tạo Ellipsoid:** Với mỗi clique tìm được, thuật toán tính ellipsoid có thể tích nhỏ nhất bao quanh tất cả các điểm trong clique đó. Ellipsoid này cung cấp tâm (điểm mẫu) và hình dạng ban đầu cho bước lấp đầy tiếp theo.
4. **Lấp đầy thành Đa diện:** Tâm và hình dạng của mỗi ellipsoid khởi tạo một vòng lặp IRIS duy nhất. Nhờ ellipsoid đã được định hướng bởi hình học cục bộ, bước lấp đầy thường chỉ cần một vòng lặp – hiệu quả hơn đáng kể so với IRIS tiêu chuẩn [?].

⁶ *Visibility graph* là đồ thị trong đó có cạnh nối hai đỉnh nếu và chỉ nếu đoạn thẳng nối chúng nằm hoàn toàn trong $\mathcal{C}_{\text{free}}$, tức là không va chạm với bất kỳ vật cản nào.

⁷ Nếu hai điểm nhìn thấy nhau, đoạn thẳng nối chúng nằm trong $\mathcal{C}_{\text{free}}$ – tính chất đặc trưng của tập lỗi.

5. **Lắp lại và hội tụ:** Quá trình được lặp lại bằng cách lấy mẫu mới từ không gian chưa được phủ cho đến khi đạt độ phủ mong muốn.

Đặc tính của Thuật toán

- **Tự động hóa đặt điểm mầm:** Khác với IRIS yêu cầu lựa chọn điểm mầm thủ công, VCC tự suy ra vị trí đặt mầm từ cấu trúc đồ thị khả kiến, giảm thiểu sự phụ thuộc vào kiến thức chuyên gia về môi trường.
- **Độ phức tạp tính toán:** Bài toán MAXCLIQUE là NP-hard trong trường hợp tổng quát; VCC dùng thuật toán tham lam lặp lại để giải gần đúng, cho kết quả tốt trong thực tế với thời gian đa thức đối với hầu hết cấu hình môi trường. Giai đoạn lấp đầy đa diện tương đương một vòng lặp IRIS với khởi tạo tốt, nên nhanh hơn IRIS tiêu chuẩn đáng kể.
- **Chất lượng phân hoạch:** VCC có xu hướng tạo ít vùng hơn để phủ cùng một không gian so với IRIS lấy mầm ngẫu nhiên, nhờ điểm mầm được đặt có chủ đích dựa trên cấu trúc hình học. Tuy nhiên, chất lượng phụ thuộc vào số lượng mẫu n và tính đại diện của đồ thị khả kiến.
- **Giới hạn:** VCC yêu cầu lấy mẫu đủ dày để xây dựng đồ thị khả kiến đại diện. Trong các không gian hẹp (narrow passages) hoặc không gian chiều cao, đây có thể là thách thức đáng kể.

Tổng kết và lựa chọn phương pháp

Ba phương pháp trình bày ở trên có những ưu nhược điểm bổ sung cho nhau, được tóm tắt trong Bảng 2.

Bảng 2. So sánh các phương pháp phân hoạch không gian an toàn.

Tiêu chí	ACD	IRIS	VCC
Phủ toàn bộ C_{free}	Có (xấp xỉ τ)	Theo seed	Tự động lặp
Yêu cầu vật cản lồi	Không	Có	Có
Tự động đặt điểm mầm	–	Không	Có
Bộ giải sử dụng	Heuristic	QP + SDP	ILP + QP + SDP
Phù hợp với	Polygon 2D	$\mathbb{R}^n$ tổng quát	$\mathbb{R}^n$ tổng quát

Trong đề án này, bước phân hoạch không gian an toàn được thực hiện bằng một trong các phương pháp trên để tạo ra họ vùng lồi $\{Q_v\}$ làm đầu vào chung cho cả hai hướng tiếp cận GCS-Bézier (Phần ??) và GCS-DMS (Phần ??). Chi tiết về tham số và kết quả phân hoạch trên các môi trường thực nghiệm được trình bày ở Chương ??.

4 Integrated Framework GCS-Bézier for Motion Planning

5 Experimental Results and Discussion

6 Conclusion and Future Work

Table 3 gives a summary of all heading levels.

Bảng 3. Table captions should be placed above the tables.

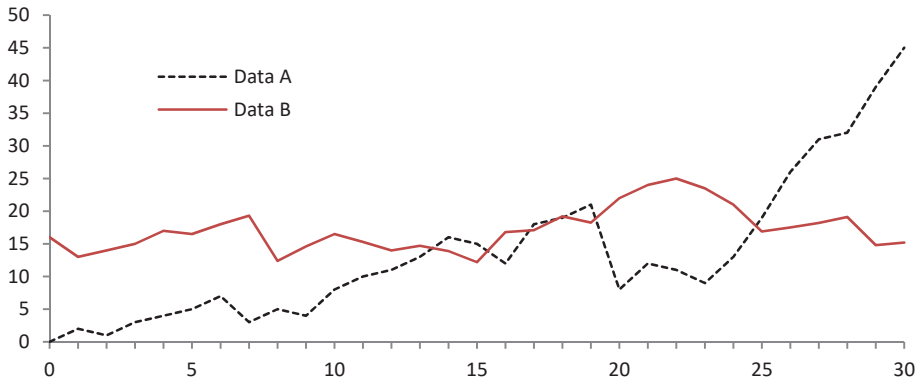
Heading level	Example	Font size and style
Title (centered)	Lecture Notes	14 point, bold
1st-level heading	1 Introduction	12 point, bold
2nd-level heading	2.1 Printing Area	10 point, bold
3rd-level heading	Run-in Heading in Bold. Text follows	10 point, bold
4th-level heading	<i>Lowest Level Heading.</i> Text follows	10 point, italic

Displayed equations are centered and set on a separate line.

$$x + y = z$$

(3)

Please try to avoid rasterized images for line-art diagrams and schemas. Whenever possible, use vector graphics instead (see Fig. 21).



Hình 21. A figure caption is always placed below the illustration. Please note that short captions are centered, while long ones are justified by the macro package automatically.

Theorem 1. *This is a sample theorem. The run-in heading is set in bold, while the following text appears in italics. Definitions, lemmas, propositions, and corollaries are styled the same way.*

Chứng minh. Proofs, examples, and remarks have the initial word in italics, while the following text appears in normal font.

For citations of references, we prefer the use of square brackets and consecutive numbers. Citations using labels or the author/year convention are also acceptable. The following bibliography provides a sample reference list with entries for journal articles [1], an LNCS chapter [2], a book [3], proceedings without editors [4], and a homepage [5]. Multiple citations are grouped [1,2,3], [1,3,4,5].

Acknowledgments. We acknowledge Ho Chi Minh City University of Technology (HCMUT), VNU-HCM for supporting this study.

Tài liệu

1. Author, F.: Article title. Journal **2**(5), 99–110 (2016)
2. Author, F., Author, S.: Title of a proceedings paper. In: Editor, F., Editor, S. (eds.) CONFERENCE 2016, LNCS, vol. 9999, pp. 1–13. Springer, Heidelberg (2016). <https://doi.org/10.1007/1234567890>
3. Author, F., Author, S., Author, T.: Book title. 2nd edn. Publisher, Location (1999)
4. Author, A.-B.: Contribution title. In: 9th International Proceedings on Proceedings, pp. 1–2. Publisher, Location (2010)
5. LNCS Homepage, <http://www.springer.com/lncs>, last accessed 2023/10/25